

Índice general

I	Aparato lógico	3
1.	Estructura de la librería <code>prop.py</code>	4
2.	Clase <code>Proposicion</code> y su subclase <code>v</code> (proposición atómica)	5
2.1.	La subclase <code>v</code> para proposiciones atómicas (i.e., para variables proposicionales)	6
3.	Valoración de proposiciones	7
3.1.	Función <code>span</code>	8
3.1.1.	Implementación	8
4.	Función <code>extension</code> que extiende una función parcial	10
4.1.	Implementación de la función <code>extension</code>	10
4.1.1.	Implementación	11
4.2.	Reducción de proposiciones NO atómicas	11
4.3.	Proposición no derivable de un conjunto de premisas	12
4.4.	... cuando llegamos a una proposición atómica (o <i>variable proposicional</i>)	13
5.	Función que refuta una proposición a partir de un conjunto de premisas	14
6.	Función <code>test</code>	15
7.	Representación de la clase <code>Proposicion</code>	16
II	Creación de preguntas de opción múltiple	17
8.	Estructura del aparato auxiliar de la librería <code>prop.py</code>	18
9.	Clase <code>Marcador</code>	19
9.1.	Implementación	19
9.1.1.	Función auxiliar <code>Siguiente (x, y)</code>	20
10.	Las clases <code>Supuesto</code> , <code>Cuestion</code> , <code>ProblemaTipo</code> y <code>ProblemaTipoProfe</code>	21
10.1.	Clase <code>Supuesto(enunciado, semantica, precond)</code>	21
10.2.	Clase <code>Cuestion(enunciado, semantica, precond)</code>	21
10.3.	Clase <code>ProblemaTipo</code>	22
10.3.1.	Implementación	22
10.4.	Ejemplo de uso	24
10.5.	Clase <code>ProblemaTipoProfe</code>	25
11.	Clase <code>ProblemaVF</code>	27
11.1.	Ejemplo de uso	27

III	Dando formato a las preguntas	28
12.	Estructura de la librería <code>FormatoPreguntas.py</code>	29
13.	Preguntas en $\text{\LaTeX}$ para usar con Auto-Multiple-Choice	30
13.1.	Variante para preguntas Verdadero/Falso	30
13.2.	Ejemplo de uso	33
14.	Preguntas en <code>xml</code> para usar con Moodle	37
14.1.	QuizMoodle, QuizMoodleLastCh, QuizVFMoodle y QuizVFMoodleLastCh	38
14.2.	MoodleMulti y MoodleMultiLastCh	40
14.2.1.	Codificación de caracteres no ASCII	41
14.3.	Ejemplo de uso de QuizMoodleLastCh y QuizVFMoodleLastCh	41
IV	Ejemplo de pregunta de opción múltiple para Econometría	43

Parte I

Aparato lógico

Capítulo 1

Estructura de la librería prop.py

```
<<Aparato lógico de la librería>>
<<Aparato auxiliar para la creación de problemas de opción múltiple>>
```

```
<<Definición de la clase Proposicion>>
<<Definición de la subclase v>>
<<Definición de alguno y unoDe>>
<<Definición de noDerivable>>
<<Definición de la función span>>
<<Definición de la función extension>>
<<Definición de la función refuta>>
<<Definición de la función test>>
```

El <<Aparato auxiliar para la creación de problemas de opción múltiple>> se desarrolla en el capítulo siguiente.

Capítulo 2

Clase `Proposicion` y su subclase `v` (proposición atómica)

Las proposiciones son expresiones formadas con números, strings y los símbolos

`&`, `|`, `-`, `v( )`, `>>`, `**`, `alguno` y `unoDe`

construidas según las siguientes reglas:

1. Si `D` es un número o un string (cadena de caracteres) entonces `v(D)` es una subclase de `Proposicion` que denominaremos “*variable proposicional*” o “*proposición atómica*”.
2. Si `P`, `Q` y `R` son proposiciones también lo son:

`P&Q` (conjunción)

`P|Q` (disyunción)

`-P` (negación)

`P>>Q` (implicación)

`P**Q` (equivalencia, es decir: *si y solo si*).

Para facilitar la elaboración expresiones, también incluimos dos operadores lógico menos habituales:

`alguno(P,Q,R)` significa que alguna de las proposiciones de la lista es verdadera y

`unoDe(P,Q,R)` significa que solo una de las proposiciones de la lista es verdadera.

Ejemplo de proposición: “Llueve o no llueve”

```
v('llueve') | -v('llueve')
```

```
v('llueve') | -v('llueve')
```

Necesitamos que las proposiciones del tipo `v(D)` sean “*hash-eables*”, para poder ser utilizadas como claves en un diccionario. Para ello necesitamos los métodos `__hash__` y `__eq__`.

Definamos la clase `Proposicion`

```
class Proposicion:
    def __init__(self, data):
        self.data = data

    def __hash__(self):
        return hash(self.data)

    def __eq__(self, another):
        return hasattr(another, 'data') and self.data == another.data
```

1. Las operaciones: conjunción, disyunción, negación, implicación y equivalencia.

Definamos como métodos dentro de la clase `Proposicion` los cinco operadores lógicos habituales. Primero aquellos que usan “*métodos mágicos*”, es decir, aquellos que tiene asociado un símbolo para llamar a las operaciones:

```
def __and__(self, other):
    return Proposicion(['and', self, other])
def __or__(self, other):
    return Proposicion(['or', self, other])
def __neg__(self):
    return Proposicion(['not', self])
def __rshift__(self, other):
    return Proposicion(['implica', self, other])
def __pow__(self, other):
    return Proposicion(['equivale', self, other])
```

Las operaciones: `unoDe` y `alguno`.

Para las dos siguientes operaciones¹ no usamos ningún “*método mágico*”. Así que definir las dentro de la clase nos obligaría a una incómoda escritura del tipo: `Proposicion.alguno(P1,P2,P3)` o `Proposicion.unoDe(P1,P2,P3)`. Para evitarlo, las definimos fuera de la clase; así podremos escribir sencillamente `alguno(P1,P2,P3)` y `unoDe(P1,P2,P3)`.

```
def alguno(X,*args):
    return Proposicion(['alguno', X] + [a for a in args])
def unoDe(X,*args):
    return Proposicion(['unoDe', X] + [a for a in args])
```

La operación: `noDerivable`.

Esta función no es una operación lógica (como si lo son las anteriores). Es un “añadido” cuya motivación es la siguiente:

El esquema típico de una pregunta es: el enunciado asume que las premisas `E1`, `E2`,... `En` son ciertas. Y se puede preguntar si `P` es consecuencia lógica de las premisas. Si lo es, `P` es `True` (es decir, `premisas ⇒ P`). Negar esto no es decir que `¬P` es consecuencia de las premisas, sino que `P` no es derivable de las premisas (es decir, `premisas ⇏ P`). Así que añadimos la operación `noDerivable`. Esto nos permite, por ejemplo, que aunque para algunos enunciados se pueda preguntar si una matriz es diagonalizable, para otros evitemos la pregunta si no es deducible del enunciado.

```
def noDerivable(X):
    return Proposicion(['noDerivable', X])
```

2.1. La subclase `v` para proposiciones atómicas (i.e., para variables proposicionales)

```
class v(Proposicion):
    def __init__(self, data):
        super().__init__(data)
```

¹no habituales, aunque convenientes pues facilitan la expresión de algunas proposiciones

Capítulo 3

Valoración de proposiciones

Una valoración es una función T que va del conjunto de proposiciones al conjunto $\{\text{True}, \text{False}\}$ (es decir, a cada proposición le asigna el valor verdadero o el valor falso) y que verifica las siguientes propiedades: si P y Q son proposiciones entonces

1. $T(P \ \& \ Q) = \text{True}$ únicamente en el caso de que $T(P) = \text{True}$ y $T(Q) = \text{True}$.
2. $T(P \mid Q) = \text{False}$ únicamente en el caso de que $T(P) = \text{False}$ y $T(Q) = \text{False}$.
3. $T(\neg P) = \text{True}$ únicamente en el caso de que $T(P) = \text{False}$.

Por tanto: $T(P \ \& \ Q) = T(P) \ \& \ T(Q)$, $T(P \mid Q) = T(P) \mid T(Q)$ y $T(\neg P) = \text{not } T(P)$.

1. $T(P \gg Q) = \text{False}$ únicamente en el caso en que $T(P) = \text{True}$ y $T(Q) = \text{False}$.
2. $T(P \ ** \ Q) = \text{True}$ únicamente en el caso en que $T(P) = T(Q)$.

Por tanto

$$T(P \ Q) = T(\neg P \mid Q)$$

y P))

$$T(P \ ** \ Q) = T((PQ) \ \& \ (Q$$

Y si $A_1, \dots, A_n$ son n proposiciones, entonces

1. $\text{alguno}(A_1, \dots, A_n)$ es True si y solo si $A_1 \mid \text{alguno}(A_2, \dots, A_n)$ es True ,
2. $\text{unoDe}(A_1, \dots, A_n)$ es True si y solo si $(A_1 \ \& \ \neg \text{alguno}(A_2, \dots, A_n) \mid (\neg A_1 \ \& \ \text{unoDe}(A_2, \dots, A_n)))$ es True ;

Por tanto

$$T(\text{alguno}(A_1, \dots, A_n)) = T(A_1 \mid \text{alguno}(A_2, \dots, A_n))$$

y

$$T(\text{unoDe}(A_1, \dots, A_n)) = T((A_1 \ \& \ \neg \text{alguno}(A_2, \dots, A_n)) \mid (\neg A_1 \ \& \ \text{unoDe}(A_2, \dots, A_n)))$$

Cualquier función F que vaya del conjunto de proposicionales atómicas (es decir, del tipo $v('algo')$) al conjunto $\{\text{True}, \text{False}\}$ se puede extender al resto proposiciones¹ formando una valoración $T = \text{span}(F)$, aplicando las reglas de arriba. Dicha extensión es única.

¹En la función `span` de Python hemos elegido el criterio de que a toda proposición `Prop` no incluida explícitamente en el diccionario `F` le asignamos el valor `False` para así completar el dominio de las proposiciones atómicas. Esta decisión ha sido completamente arbitraria, y se ha incluido para que efectivamente el dominio de `span` incluya todas las proposiciones atómicas (i.e., variables proposicionales). En cualquier caso `span` se ha programado a modo de ilustración, pues no la emplearemos para nada.

Vamos a implementar como funciones los diccionarios de Python, pues los diccionarios son una colección de pares 'Etiqueta':valor donde a cada etiqueta solo se le asigna un valor único. Por ejemplo, si definimos el siguiente diccionario {'a':1 , 'a':0 , 'b':2}, obtenemos un diccionario con dos elementos, donde a 'a' se le ha asignado solo uno de los dos valores introducidos (en concreto el último).

```
print( {'a':1 , 'a':0 , 'b':2} )
```

```
{'a': 0, 'b': 2}
```

3.1. Función span

Función `span(F, Prop)` indica si `Prop` es `True` o es `False` a partir de la función parcial `F`.

- `F`: es un diccionario (función parcial) que asigna valores `True` o `False` a varias proposiciones atómicas.
- `Prop`: es una Proposición .

Así obtenemos los siguientes resultados lógicos:

```
F = { v('llueve'):True }
span(F, v('llueve')) >> -v('llueve') )
```

False

y por otra parte

```
span(F, -v('llueve')) >> v('llueve') )
```

True

3.1.1. Implementación

Si una proposición `Prop` (de tipo atómico) está en la función parcial `F` y además es verdadera, `span` devuelve `True`. En caso contrario devuelve `False` (tanto si `Prop` no está en `F` como si lo está pero es `False`). Si `Prop` no es de tipo atómico, entonces se aplican las reglas para reducir `Prop` a proposiciones de tamaño más pequeño.

```
def span(F, Prop):
    if type(Prop) == type(v('')):
        return (Prop in F) and F[Prop]
    <<Aplicación de las reglas para reducir el tamaño de Prop>>

    elif Prop.data[0] == 'and':
        return span(F, Prop.data[1]) and span(F, Prop.data[2])

    elif Prop.data[0] == 'or':
        return span(F, Prop.data[1]) or span(F, Prop.data[2])

    elif Prop.data[0] == 'not':
        return not span(F, Prop.data[1])

    elif Prop.data[0] == 'implica':
        return (not span(F, Prop.data[1])) or span(F, Prop.data[2])

    elif Prop.data[0] == 'equivale':
        return span(F, Prop.data[1]) == span(F, Prop.data[2])

    elif Prop.data[0] == 'alguno':
        return span(F, Prop.data[1]) or span(F, alguno(Prop.data[2:])) if len(Prop.data)>2\
        else span(F, Prop.data[1])

    elif Prop.data[0] == 'unoDe':
        A = Prop.data[1]
        if len(Prop.data)==2:
            return span(F, A)
        else:
```

```
B = Proposicion(['alguno'] + Prop.data[2:])
C = Proposicion(['unoDe'] + Prop.data[2:])
return span(F, (A&B) | (~A&C) )
```

Capítulo 4

Función `extension` que extiende una función parcial

Ahora queremos definir una función de Python (que denominamos `extension`) que, dadas una lista `objetivo` y un diccionario con `valores`, para una colección de proposiciones atómicas haga crecer (extienda) el diccionario `valores` con nuevas proposiciones atómicas, y sus correspondientes valores, necesarios para que se cumplan los pares contenidos en la lista `objetivo`; o bien que devuelva el diccionario vacío `{}` en caso de que sea imposible hacer cumplir dicha lista `objetivo`.

La diferencia entre la lista `objetivo` y los `valores` es que en `objetivo` se asignan valores a proposiciones (arbitrarias) pero en `valores` solo se asignan valores a *proposiciones atómicas*.

El diccionario extendido es un *modelo lógico* en el que se cumplen la lista `objetivo` y la función parcial `valores`. Ello no quiere decir que no puedan existir otros modelos que consigan lo mismo.

Ejemplo:

```
print( extension( [ (v('llueve') >> -v('llueve')) , True] , {v('basilisco') : True}) )
```

```
{v('basilisco'): True, v('llueve'): False}
```

Ejemplo:

```
print( extension( [ (v('llueve'),True), (-v('llueve'),True)], {v('basilisco') : True}) )
```

```
{}
```

4.1. Implementación de la función `extension`

Función `extension(objetivo, valores, premisas=[])`

- `objetivo`: es una lista de pares (P, VF) donde P es una proposición y VF es `True` o `False`
- `valores`: es un diccionario (función parcial) que asigna valores `True` o `False` a variables proposicionales
- `premisas`: es la lista de premisas a partir de la cual se deduce si una proposición es derivable o no (es decir, es solo para usar la operación `noDerivable`)

La función `extension` tiene dos posibles salidas: o bien devuelve un modelo lógico M que extiende a `valores` de tal forma que $\text{span}(M, P) = VF$ para cada par (P, VF) de `objetivo`, o el diccionario vacío `{}` si no existe tal modelo M . Es decir, busca una forma de dar valores a las proposiciones atómicas de modo que todas las proposiciones contenidas en el `objetivo` alcancen el valor deseado.

4.1.1. Implementación

Si la lista `objetivo` no tiene pares, con la propia función `valores`, hemos acabado.

La definición de la función `extension` es recursiva. La mecánica es la siguiente: se sustituye el proposición del primer objetivo por proposiciones de menor tamaño¹, y entonces se llama a si misma. Este procedimiento reduce las proposiciones hasta obtener proposiciones atómicas.

Así pues, llamamos `Obj` al primer objetivo de la lista, `P` a la correspondiente proposición y `VF` a su valor objetivo. Por tanto `Obj = (P,VF)` es el primer par de la lista `objetivo`.

Comprobamos que `P.data` es una `list` y tratamos los distintos casos de reducción de predicado.

```
def extension(objetivo, valores, premisas=[]):
    if objetivo == []:
        return valores

    Obj = objetivo[0]
    P = Obj[0]
    VF = Obj[1]
    if isinstance(P.data,list):
        <<Reducción de una proposición de tipo "not">>
        <<Reducción de una proposición de tipo "and">>
        <<Reducción de una proposición de tipo "or">>
        <<Reducción de una proposición de tipo "implica">>
        <<Reducción de una proposición de tipo "equivale">>
        <<Reducción de una proposición de tipo "alguno">>
        <<Reducción de una proposición de tipo "unoDe">>
        <<Proposición de tipo "noDerivable">>
        <<Proposición atómica v>>
```

4.2. Reducción de proposiciones NO atómicas

1. Reducción de una proposición de tipo “not”.

En el caso de que `Obj = (-A,True)`, nos vale la misma función que obtenemos si reemplazamos `Obj` por `(A,False)`. Y en el caso de que `Obj = (-A,False)`, nos vale la misma función si reemplazamos `Obj` por `(A,True)`.

```
if P.data[0] == 'not' :
    A = P.data[1]
    return extension( [(A, not VF)] + objetivo[1:], valores, premisas )
```

2. Reducción de una proposición de tipo “and”.

En el caso de que `Obj=(A & B, True)`, nos vale la misma función que obtenemos si reemplazamos `Obj` por `(A,True), (B,True)`. Y en el caso de que `Obj=(A & B, False)`, vale tanto la función que obtenemos de reemplazar `Obj` por `(A,False)` como la que obtenemos al reemplazar `Obj` por `(B,False)`.

Si la primera alternativa resulta en una extensión vacía se devuelve la segunda.

```
elif P.data[0] == 'and':
    A = P.data[1]
    B = P.data[2]
    if (VF):
        return extension([(A,True), (B,True)] + objetivo[1:], valores, premisas)
    else:
        r = extension([(A,False)] + objetivo[1:],valores,premisas)
        return extension([(B,False)]+objetivo[1:],valores,premisas) if not r else r
```

3. Reducción de una proposición de tipo “or”.

En el caso de que `Obj=( A|B, False)`, nos vale la misma función que obtenemos si reemplazamos `Obj` por `(A,False), (B,False)`. Y en el caso de que `Obj=( A&B, True)`, vale tanto la función que obtenemos de reemplazar `Obj` por `(A,True)` como la que obtenemos al reemplazar `Obj` por `(B,True)`.

¹Una lista de símbolos más corta

Si la primera alternativa resulta en una extensión vacía se devuelve la segunda.

```

elif P.data[0] == "or":
    A = P.data[1]
    B = P.data[2]
    if not VF:
        return extension([(A,False),(B,False)] + objetivo[1:], valores, premisas)
    else:
        r = extension([(A,True)] + objetivo[1:], valores, premisas)
        return extension([(B,True)]+objetivo[1:],valores, premisas) if not r else r

```

4. Reducción de una proposición de tipo “implica”.

En el caso de que $\text{Obj} = (A \gg B, VF)$, nos vale la misma función que obtenemos si emplazamos Obj por $(\neg A \vee B, VF)$.

```

elif P.data[0] == "implica":
    A = P.data[1]
    B = P.data[2]
    return extension([(¬A|B, VF)] + objetivo[1:], valores, premisas)

```

5. Reducción de una proposición de tipo “equivale”.

En el caso de que $\text{Obj} = (A \leftrightarrow B, VF)$, nos vale la misma función que obtenemos si remplazamos Obj por $((A \gg B) \& (B \gg A), VF)$.

```

elif P.data[0] == "equivale":
    A = P.data[1]
    B = P.data[2]
    return extension([(A>>B) & (B>>A), VF]) + objetivo[1:], valores, premisas)

```

6. Reducción de una proposición de tipo “alguno”.

En caso de que $\text{Obj} = (\text{alguno}(A), VF)$ (i.e., la lista de alternativas solo contiene la proposición A) nos vale la misma función que (A, VF) . Si por el contrario $\text{Obj} = (\text{alguno}(A_1, \dots, A_n), VF)$, con $n > 1$, nos vale la misma función que obtenemos si remplazamos Obj por $(A_1 \vee \text{alguno}(A_2, \dots, A_n), VF)$ (i.e., la primera o alguna de las demás).

```

elif P.data[0] == "alguno":
    A = P.data[1]
    if len(P.data)==2:
        return extension([(A,VF)] + objetivo[1:], valores, premisas)
    else:
        B = Proposicion(["alguno"] + P.data[2:])
        return extension([(A|B, VF)] + objetivo[1:], valores, premisas)

```

7. Reducción de una proposición de tipo “unoDe”.

En caso de que $\text{Obj} = (\text{unoDe}(A), VF)$ (i.e., la lista de alternativas solo contiene la proposición A) nos vale la misma función que (A, VF) . Si por el contrario $\text{Obj} = (\text{unoDe}(A_1, \dots, A_n), VF)$, con $n > 1$, nos vale la misma función que obtenemos si remplazamos Obj por $(A_1 \& \neg \text{alguno}(A_2, \dots, A_n) \vee \neg A_1 \& \text{unoDe}(A_2, \dots, A_n), VF)$.

```

elif P.data[0] == "unoDe":
    A = P.data[1]
    if len(P.data)==2:
        return extension([(A,VF)] + objetivo[1:], valores, premisas)
    else:
        B = Proposicion(["alguno"] + P.data[2:])
        C = Proposicion(["unoDe"] + P.data[2:])
        return extension([(A&¬B) | (¬A&C), VF]) + objetivo[1:], valores, premisas)

```

4.3. Proposición no derivable de un conjunto de premisas

La función `extension` tiene un tercer argumento (`premisas`) que es un “añadido” para decidir si una proposición es `noDerivable` de la lista de `premisas`.

Derivable significa que la proposición es consecuencia lógica de las **premisas**, por consiguiente es imposible que sea falso si las **premisas** son ciertas.

```
elif P.data[0] == "noDerivable":
    A = P.data[1]
    valoresExt = valores.copy()
    valoresExt[v(repr(P))] = VF

    derivable = not extension([(A,False)] + premisas, {}, premisas)

    return extension(objetivo[1:],valores2,premisas) if not derivable == VF else {}
```

4.4. ... cuando llegamos a una proposición atómica (o *variable proposicional*)

En el caso de que $\text{Obj} = (v(D), VF)$, es cuando el algoritmo recursivo pasará a intentar un nuevo objetivo (extendiendo el diccionario **valores** si fuera necesario), o bien parará por ser el par objetivo $(v(D), VF)$ imposible.

Primero se comprueba si el par $(v(D), VF)$ ya estaba en el diccionario **valores**, es decir, si **valores** contiene la proposición atómica $v(D)$ y con el valor **VF**. En tal caso, se “salta” al siguiente objetivo (es decir se llama al **extension** con el mismo diccionario **valores** pero quitando de la lista **objetivo** el primer elemento. Si $v(D)$ pertenece al diccionario **valores** pero **con un valor distinto** a **VF**, quiere decir que el objetivo es imposible de alcanzar, por lo que se devuelve un diccionario vacío, y la recursión ha terminado.

Si **P** es una proposición atómica que no estaba en el diccionario **valores**, entonces creamos un nuevo diccionario extendido (**valoresExt**) que incluya **P** y su valor **VF**; y entonces llamamos a **extension** pero quitando el primer objetivo de la lista y usando un diccionario “extendido” (**valoresExt**). Para evitar los efectos colaterales de modificar un diccionario, modificamos una copia

```
else:
    if P in valores:
        return extension(objetivo[1:], valores, premisas) if valores[P]==VF else {}
    else:
        valoresExt = valores.copy()
        valoresExt[P] = VF
        return extension(objetivo[1:], valoresExt, premisas)
```

Capítulo 5

Función que refuta una proposición a partir de un conjunto de premisas

Refutar una proposición, consiste en asignar un valor a cada variable proposicional (cada proposición atómica), de tal modo que el resultado sea falso.

La función `refuta` construye un modelo `M` (asignación de valores a las proposiciones atómicas) de forma que `span(M,P)=False` y `span(M,h)=True` para todo `h` en `premisas`.

Si esto no es posible refutar `P` a partir de las `premisas`, entonces la función `refuta` devuelve un modelo (un diccionario) vacío; esto quiere decir que la proposición `P` es irrefutable, es decir, $\text{=premisas} \Rightarrow \text{=P}$. Es decir, si se refuta es que no es derivable de las premisas (no es consecuencia lógica de las premisas).

```
def refuta(P, premisas=[]):  
    h=[(Q,True) for Q in premisas]  
    return extension([(P,False)]+h, {}, h)
```

Capítulo 6

Función test

Función `test(P,premisas)`

- `P`: es una proposición o cualquiera de los valores `True`, `False`.
- `premisas`: es una lista de proposiciones

La función `test` se utiliza tanto para saber si se da una precondition, como para saber el resultado correcto de una cuestión.

En el caso de que `P` sea una proposición, sólo devuelve `True` si es imposible refutarlo a partir de las `premisas`, es decir, es imposible que `P` sea falso si las `premisas` son verdad ($\text{premisas} \Rightarrow P$, i.e., `P` es derivable).

```
def test(P,premisas=[]):  
    if P == True:  
        return True  
    if P == False:  
        return False  
    else:  
        return True if not refuta(P, premisas) else False
```

Capítulo 7

Representación de la clase Proposicion

```
def __repr__(self):
    if isinstance(self.data, list):
        if self.data[0] == "and":
            return repr(self.data[1]) + " & " + repr(self.data[2])
        if self.data[0] == "or":
            return repr(self.data[1]) + " | " + repr(self.data[2])
        if self.data[0] == "not":
            return "-" + repr(self.data[1])
        if self.data[0] == "implica":
            return repr(self.data[1]) + " >> " + repr(self.data[2])
        if self.data[0] == "equivale":
            return repr(self.data[1]) + " ** " + repr(self.data[2])
        if self.data[0] == "unoDe":
            return "unoDe(' + ', '.join([repr(x) for x in self.data[1:]]) + ')"
        if self.data[0] == "noDerivable":
            return "noDerivable("+repr(self.data[1]) + ")"
    else:
        return 'v(' + repr(self.data) + ')'
```

Parte II

Creación de preguntas de opción múltiple

Capítulo 8

Estructura del aparato auxiliar de la librería `prop.py`

```
<<Definición de la clase Marcador>>  
<<Definición de la clase Supuesto>>  
<<Definición de la clase Cuestion>>  
<<Definición de la clase ProblemaTipo>>  
<<Definición de la clase ProblemaTipoProfe>>  
<<Definición de la clase ProblemaVF>>
```

Capítulo 9

Clase Marcador

Usaremos la clase `Marcador` como una herramienta auxiliar para construir el árbol con todas las combinaciones posibles entre una conjunto de enunciados y un conjunto de cuestiones. Posteriormente se descartarán de dicho árbol las combinaciones de enunciados y cuestiones que no tengan solución.

Clase `Marcador`

- `objetivo` es una lista de números naturales mayores que cero

La clase `Marcador` es un iterador. Si `objetivo=[n_1,...,n_k]` genera todas las listas de la forma `[m_1,...,m_k]` tales que cada `m_i` es un número natural menor que `n_i`

```
for i in Marcador([2,3,1]):  
    print(i)
```

```
[0, 0, 0]  
[0, 1, 0]  
[0, 2, 0]  
[1, 0, 0]  
[1, 1, 0]  
[1, 2, 0]
```

9.1. Implementación

El atributo `self.data` es la lista de números que hemos dado como argumento. En el ejemplo anterior, `[2,3,1]`.

Para construir un iterador necesitamos los métodos `__iter__` y `__next__`.

```
class Marcador:  
    def __init__(self, data):  
        self.data = data  
    def __iter__(self):  
        self.p = [0 for x in self.data]  
        return self  
    def __next__(self):  
        def Siguiente(x,y):  
            if x == [] :  
                return []  
            s = Siguiente(x[1:],y[1:])  
            if s == []:  
                if x[0]+1 == y[0]:  
                    return []  
                else:  
                    return [x[0]+1] + [0 for i in x[1:]]  
            else:  
                return [x[0]] + s  
        if self.p == []:  
            raise StopIteration
```

```

n = self.p
self.p = Siguiente(self.p, self.data)
return n

```

El atributo `self.p` contiene la lista que entregará `__next__` en la siguiente invocación al método; y el método `__iter__` genera la semilla de dicha lista, en particular dicha semilla es una lista de ceros.

```

def __iter__(self):
    self.p = [0 for x in self.data]
    return self

```

`__next__` devuelve `self.p` en su estado actual (`n`), y coloca en `self.p` la lista siguiente. Si `self.p` es vacía detiene la iteración.

```

def __next__(self):
    <<Definición de la función local Siguiente>>
    if self.p == []:
        raise StopIteration
    n = self.p
    self.p = Siguiente(self.p, self.data)
    return n

```

Dentro del método `__next__` definimos de manera recursiva la función auxiliar local `S`, que calcula la lista siguiente.

9.1.1. Función auxiliar Siguiente (x, y)

- `x`: lista de números naturales $[m_1 \dots m_r]$ tales que $m_i < n_i$ (es la lista de la que queremos calcular la siguiente).
- `y`: lista de números naturales $[n_1 \dots n_r]$ mayores que cero (lista de topes).

Devuelve $[k_1 \dots k_r]$ la siguiente lista a $[m_1 \dots m_r]$ (según el orden lexicográfico) tal que $k_i < n_i$. En caso de no existir tal lista devuelve `[]`.

```

def Siguiente(x,y):
    if x == [] :
        return []
    s = Siguiente(x[1:],y[1:])
    if s == []:
        if x[0]+1 == y[0]:
            return []
        else:
            return [x[0]+1] + [0 for i in x[1:]]
    else:
        return [x[0]] + s

```

Capítulo 10

Las clases `Supuesto`, `Cuestion`, `ProblemaTipo` y `ProblemaTipoProfe`

10.1. Clase `Supuesto(enunciado, semantica, precond)`

Clase `Supuesto(enunciado, semantica, precond)`

Es un constructor que sirve para almacenar tres aspectos (atributos) del supuesto de un problema.

enunciado string que presenta la supuesto en lenguaje humano.

semantica proposición que traduce parcialmente el significado lógico del **enunciado** (contiene lo necesario para deducir las respuestas).

precond es una proposición o bien cualquiera de los valores `True`, `False`. El `Supuesto` será incluido en el problema solo si la evaluación de **precond** es `True`.

```
class Supuesto:
    def __init__(self, enunciado, semantica, precond=True):
        self.e = enunciado
        self.s = semantica
        self.p = precond
```

10.2. Clase `Cuestion(enunciado, semantica, precond)`

Clase `Cuestion(enunciado, semantica, precond)`

Es un constructor que sirve para almacenar tres aspectos (atributos) de una cuestión de un problema.

enunciado string que presenta la cuestión en lenguaje humano.

semantica proposición P tal que `test(P, semántica del conjunto de supuestos)` devuelve la respuesta correcta.

precond es una proposición o bien cualquiera de los valores `True`, `False`. La `Cuestion` será incluida en el problema solo si la evaluación de **precond** es `True`.

```
class Cuestion:
    def __init__(self, enunciado, semantica, precond=True, exp=""):
        self.e = enunciado
        self.s = semantica
        self.p = precond
        self.x = exp
```

10.3. Clase ProblemaTipo

Clase ProblemaTipo (supuestos_y_cuestiones)

`supuestos_y_cuestiones` es una lista de listas de strings, Supuesto-s, o Cuestion-es.

Cada lista representa un conjunto de opciones excluyentes. Para mayor comodidad, en el caso de que una de esas listas consista en un único objeto (una única opción), también se permite escribir directamente el objeto sin encerrarlo en una lista, es decir, la lista `supuestos_y_cuestiones` también admite objetos de tipo: string, Supuesto o Cuestion.

10.3.1. Implementación

La clase ProblemaTipo es un iterador que genera todas las versiones aceptables de un problema. Cada variante resulta de tomar una de las opciones de cada una de las listas en `supuestos_y_cuestiones`, pero si la precondición de alguna de las opciones elegidas es False la correspondiente variante del problema es rechazada.

- El atributo `self.e` contiene la lista `supuestos_y_cuestiones`. Cada elemento de `supuestos_y_cuestiones` es a su vez una lista de opciones (excluyentes) correspondiente a cada componente (cada parte) del texto del problema. La combinación de distintas opciones crea el texto completo de una variante del problema.

El método `__iter__` inicializa el iterador. El método `__next__` obtiene la siguiente variante del problema, pero asegurándose de que las precondiciones son satisfechas (si no la variante es desechada).

```
class ProblemaTipo:
    def __init__(self, supuestos_y_cuestiones):
        self.e = supuestos_y_cuestiones

    def __iter__(self):
        self.l = [x if isinstance(x,list) else [x] for x in self.e]
        self.long = len(self.l)
        self.i = iter(Marcador([len(x) for x in self.l]))
        self.c = 0
        return self

    def __next__(self):
        self.c += 1
        while True:
            try:
                variante = next(self.i)
            except StopIteration:
                raise StopIteration

            enunciado = ""
            hipotesis = []
            cuestiones = []

            for n in range(self.long+1):
                if n == self.long:
                    return (str(self.c), enunciado, cuestiones)

                componente = self.l[n][variante[n]]
                if isinstance(componente, str):
                    enunciado = enunciado + componente

                elif isinstance(componente, Supuesto):
                    if test(componente.p, hipotesis):
                        enunciado = enunciado + componente.e
                        hipotesis = hipotesis + [componente.s]
                    else:
                        print('\n Supuesto: ' + str(componente.e) \
                              + ' rechazado por ' + str(componente.p) + '\n')
                        break

                elif isinstance(componente, Cuestion):
                    if test(componente.p, hipotesis):
                        cuestiones = cuestiones + \
                            [(componente.e,(True if test(componente.s, hipotesis) else False),1,componente.x)]
                    else:
                        cuestiones = cuestiones + \
                            [(componente.e,'rechazada por ' + str(componente.p),0,componente.x)]
                        print('\n Cuestion: ' + str(componente.e) \
```

```
        + ' rechazada por ' + str(componente.p) + '\n')
    break
```

El atributo `self.l` es la lista `self.e` normalizada para que todos sus objetos sean listas; `self.long` es el número de componentes que constituyen el texto de cada variante. `self.i` es el iterador y `self.c` un contador.

```
def __iter__(self):
    self.l = [x if isinstance(x,list) else [x] for x in self.e]
    self.long = len(self.l)
    self.i = iter(Marcador([len(x) for x in self.l]))
    self.c = 0
    return self
```

La generación de **variantes** de un **ProblemaTipo** se produce dentro de un bucle que solo se detiene cuando el intento (`try`) de iterar una vez más, `next(self.i)` falla por haberse agotado todas las variantes.

`self.long` guarda el número de elementos (es decir, de cadenas, **Supuesto-s** y **Cuestion-es**) que constituyen el texto del problema.

En cada iteración 1) **enunciado**, **hipotesis** y **cuestiones** comienzan estando vacías; y 2) se recorren uno a uno los componentes de la variante que se intenta construir.

Una vez se han recorrido todos los componentes se devuelve la lista: (`str(self.c)`, **enunciado**, **cuestiones**) que constituye la variante concreta del problema que se ha generado en esa iteración. Cada tipo de **componente** es tratado de una forma distinta.

```
def __next__(self):
    self.c += 1
    while True:
        try:
            variante = next(self.i)
        except StopIteration:
            raise StopIteration

        enunciado = ""
        hipotesis = []
        cuestiones = []

        for n in range(self.long+1):
            if n == self.long:
                return (str(self.c), enunciado, cuestiones)

            componente = self.l[n][variante[n]]
            if isinstance(componente, str):
                enunciado = enunciado + componente

            elif isinstance(componente, Supuesto):
                if test(componente.p, hipotesis):
                    enunciado = enunciado + componente.e
                    hipotesis = hipotesis + [componente.s]
                else:
                    print('\n Supuesto: ' + str(componente.e) \
                        + ' rechazado por ' + str(componente.p) + '\n')
                    break

            elif isinstance(componente, Cuestion):
                if test(componente.p, hipotesis):
                    cuestiones = cuestiones + \
                        [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
                else:
                    cuestiones = cuestiones + \
                        [(componente.e, 'rechazada por ' + str(componente.p), 0, componente.x)]
                    print('\n Cuestion: ' + str(componente.e) \
                        + ' rechazada por ' + str(componente.p) + '\n')
                    break
```

- Cuando el **componente** es una cadena, se considera dicha **componente** es parte del **enunciado** del problema, por lo que consecuentemente es añadido al mismo (es decir, se concatena al string **enunciado**).

- Cuando el componente es un Supuesto, se testea la **precondicion**. Si `componente.p` es True o no es refutable a partir de las **hipotesis** entonces `componente.e` es añadido al enunciado y la proposición `componente.s` es añadida a **hipotesis**. En caso contrario la variante del problema es completamente descartada y se imprime en la pantalla el motivo.
- Cuando el componente es una Cuestion, se testea la **precondicion**. Si `componente.p` es True o no es refutable a partir de las **hipotesis** entonces el par (`componente.e`, `True` `!=` `False` dependiendo de lo que corresponda) es añadido a la lista de **cuestiones**. En caso contrario la variante del problema es completamente descartada y se imprime en la pantalla el motivo.

```

if isinstance(componente, str):
    enunciado = enunciado + componente

elif isinstance(componente, Supuesto):
    if test(componente.p, hipotesis):
        enunciado = enunciado + componente.e
        hipotesis = hipotesis + [componente.s]
    else:
        print('\n Supuesto: ' + str(componente.e) \
              + ' rechazado por ' + str(componente.p) + '\n')
        break

elif isinstance(componente, Cuestion):
    if test(componente.p, hipotesis):
        cuestiones = cuestiones + \
            [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
    else:
        cuestiones = cuestiones + \
            [(componente.e, 'rechazada por ' + str(componente.p), 0, componente.x)]
        print('\n Cuestion: ' + str(componente.e) \
              + ' rechazada por ' + str(componente.p) + '\n')
        break

```

10.4. Ejemplo de uso

```

p = ProblemaTipo( \
    [
        "Considere ",
        [
            Supuesto("A ", v("A")),
            Supuesto("B ", v("B")),
        ],
        [
            Supuesto("y que A implica C. ", v("A") >> v("C")),
            Supuesto("y que B equivale a D. ", v("B") ** v("D")),
        ],
        "Conteste: ",
        [
            Cuestion("¿Se da C?", v("C")),
            Cuestion("¿Se da D?", v("D")),
            Cuestion("¿Se da E?", v("E"), v("D")),
        ],
    ],
)

for i in p:
    print(i)

```

```

('1', 'Considere A y que A implica C. Conteste: ', [( '¿Se da C?', True, 1, '')])
('2', 'Considere A y que A implica C. Conteste: ', [( '¿Se da D?', False, 1, '')])

```

```

Question: ¿Se da E? rechazada por v('D')

```

```

('3', 'Considere A y que B equivale a D. Conteste: ', [( '¿Se da C?', False, 1, '')])
('4', 'Considere A y que B equivale a D. Conteste: ', [( '¿Se da D?', False, 1, '')])

```

```

Question: ¿Se da E? rechazada por v('D')

```

```

('5', 'Considere B y que A implica C. Conteste: ', [( '¿Se da C?', False, 1, '')])
('6', 'Considere B y que A implica C. Conteste: ', [( '¿Se da D?', False, 1, '')])

```

Question: ¿Se da E? rechazada por v('D')

```
('7', 'Considere B y que B equivale a D. Conteste: ', [['¿Se da C?', False, 1, '']])
('8', 'Considere B y que B equivale a D. Conteste: ', [['¿Se da D?', True, 1, '']])
('9', 'Considere B y que B equivale a D. Conteste: ', [['¿Se da E?', False, 1, '']])
```

10.5. Clase ProblemaTipoProfe

Clase ProblemaTipoProfe (supuestos_y_cuestiones)

supuestos_y_cuestiones es una lista de listas de strings, Supuesto-s, o Cuestion-es.

Para que sea más fácil la revisión y mantenimiento de cada pregunta, esta versión muestra todas las cuestiones posibles para cada enunciado. También muestra las cuestiones excluidas y el motivo de la exclusión. Tan solo cambia el tratamiento de las cuestiones en el método `__next__`:

```
class ProblemaTipoProfe:
    def __init__(self, supuestos_y_cuestiones):
        <<Metodo auxiliar para juntar cuestiones en formato para profe>>
        self.e = CuestionesJuntas(supuestos_y_cuestiones)

    <<Método __iter__ para la clase ProblemaTipo>>
    <<Método __next__ para la clase ProblemaTipoProfe>>
```

Por tanto este trozo del código es igual que para el ProblemaTipo salvo la última parte

```
def __next__(self):

    self.c += 1
    while True:
        try:
            variante = next(self.i)
        except StopIteration:
            raise StopIteration

        enunciado = ""
        hipotesis = []
        cuestiones = []

        for n in range(self.long+1):
            if n == self.long:
                return (str(self.c), enunciado, cuestiones)

        componente = self.l[n][variante[n]]
        if isinstance(componente, str):
            enunciado = enunciado + componente

        elif isinstance(componente, Supuesto):
            if test(componente.p, hipotesis):
                enunciado = enunciado + componente.e
                hipotesis = hipotesis + [componente.s]
            else:
                print('\n Supuesto: ' + str(componente.e) \
                    + ' rechazado por ' + str(componente.p) + '\n')
                break

        elif isinstance(componente, Cuestion):
            if test(componente.p, hipotesis):
                cuestiones = cuestiones + \
                    [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
            else:
                cuestiones = cuestiones + \
                    [(componente.e, 'rechazada por ' + str(componente.p), 0)]
```

que cambia en su última parte

```
if isinstance(componente, str):
    enunciado = enunciado + componente

elif isinstance(componente, Supuesto):
    if test(componente.p, hipotesis):
        enunciado = enunciado + componente.e
```

```
        hipotesis = hipotesis + [componente.s]
    else:
        print('\n Supuesto: ' + str(componente.e) \
              + ' rechazado por ' + str(componente.p) + '\n')
        break

elif isinstance(componente, Question):
    if test(componente.p, hipotesis):
        cuestiones = cuestiones + \
            [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
    else:
        cuestiones = cuestiones + \
            [(componente.e, 'rechazada por ' + str(componente.p), 0)]
```

Capítulo 11

Clase ProblemaVF

Para crear cuestiones del tipo Verdadero/Falso.

```
from random import sample
class ProblemaVF():
    def __init__(self, enunciado, cuestiones, NumPreguntas):
        self.e = enunciado
        self.c = cuestiones
        self.NumPreguntas = NumPreguntas

    def __iter__(self):
        self.contador = 0
        return self

    def __next__(self):
        cuestiones = sample(self.c, self.NumPreguntas)
        self.contador += 1
        return (str(self.contador), self.e, cuestiones)
```

11.1. Ejemplo de uso

Cuatro exámenes con dos preguntas tipo test cada uno.

```
enunciado = "Marque las verdaderas:"

banco = [
    ("Todo alumno que va a clase aprueba", False),
    ("Todo alumno que suspende va a clase", False),
    ("Todo alumno que sabe aprueba", True),
    ("Si aprueban todos eres buen profesor", False),
    ("Si suspenden todos eres mal profesor", False),
    ("Si suspenden todos es frustrante", True),]

p = ProblemaVF (enunciado, banco, 3)      # tres preguntas por variante
p0 = iter(p)

for i in range(4):                       # cuatro variantes
    print(next(p0))
```

```
('1', 'Marque las verdaderas:', [('Si suspenden todos es frustrante', True), ('Todo alumno que suspe
('2', 'Marque las verdaderas:', [('Todo alumno que sabe aprueba', True), ('Si aprueban todos eres bu
('3', 'Marque las verdaderas:', [('Si aprueban todos eres buen profesor', False), ('Todo alumno que
('4', 'Marque las verdaderas:', [('Si suspenden todos es frustrante', True), ('Si aprueban todos ere
```

Parte III

Dando formato a las preguntas

Capítulo 12

Estructura de la librería FormatoPreguntas.py

```
from prop import *

<<Metodo auxiliar para juntar cuestiones en formato para profe>>
<<Formato AMC>>
<<Formato AMC Verdadero/Falso>>
<<Formato AMC version Profe>>
<<Formato AMC con lastchoice>>
<<Formato AMC y multicolumna>>
<<Formato AMC y multicolumna version Profe>>
<<Formato AMC con lastchoice y multicolumna>>

<<Método codchar para codificar caracteres no ASCII en \LaTeX{}>>

<<Quiz para Moodle>>
<<Quiz para Moodle con valores>>
<<Quiz para Moodle versión Profe>>
<<Quiz para Moodle versión Profe con valores>>
<<Quiz VF para Moodle>>
<<Formato Moodle de preguntas de elección múltiple>>
<<Formato Moodle de preguntas de elección múltiple para el Profe>>

<<Quiz para Moodle con LastChoice>>
<<Quiz para Moodle con LastChoice con valores>>
<<Quiz VF para Moodle con LastChoice>>
<<Formato Moodle de preguntas de elección múltiple con LastChoice>>

<<Quiz para Jupyter>>
```

Capítulo 13

Preguntas en L^AT_EX para usar con Auto-Multiple-Choice

Recorremos con un bucle la lista `cuestiones` de manera que en cada paso `c` es una nueva cuestión. Si `c[2]` es cierta (si se verifica la precondition necesaria para formular la cuestión), añadimos dicha cuestión en la cadena de cuestiones `s` del siguiente modo: si la cuestión es verdadera (si `c[1]==True`) escribimos `\correctchoice{}`; y si no, escribimos `\wrongchoice{}`. Seguidamente escribimos el enunciado de la cuestión, `c[0]`, cerramos el comando de L^AT_EX y saltamos de línea de texto, `}\n`.

```
for c in cuestiones:
    if c[2]:
        s = s + ' ' + '\n'
        + ('\\correctchoice{' if c[1] else '\\wrongchoice {'} + c[0] + '}\n'
```

En el caso de las preguntas en versión para el profe, si `c[2]` no es cierta, añadimos dicha cuestión en la cadena `ex` de *cuestiones excluidas*, seguida de un punto y coma y la proposición `c[1]` correspondiente a dicha cuestión. Así, en el formato AMC para el profesor aparecerá, en la parte de explicación de cada pregunta, el listado de preguntas excluidas dado el enunciado de esa pregunta.

```
for c in cuestiones:
    if c[2]:
        s = s + ' ' + '\n'
        + ('\\correctchoice{' if c[1] else '\\wrongchoice {'} + c[0] + '}\n'
    else:
        ex = ex + ' question: ' + c[0] + '; ' + c[1] + '\n'
```

Cada variante tendrá una opción correcta si también añadimos la opción: “ninguna de las anteriores es correcta”. El comando `\lastchoices\n` asegura que ésta sea la última. Indicamos que es incorrecta si alguna de las opciones en la lista `cuestiones` es `True`, o que ésta es la opción correcta en caso contrario.

```
s = s + ' ' + '\n' + '\\lastchoices\n'
s = s + ' ' + '\n'
+ ('\\wrongchoice {' if any([c[1] for c in cuestiones]) else '\\correctchoice{'})\n
+ OpcPorDefecto + '}\n'
```

13.1. Variante para preguntas Verdadero/Falso

Las preguntas de ejercicios del tipo Verdadero/Falso no tienen pre-condición, por tanto `c[2]` no existe.

```
for c in cuestiones:
    s = s + (" " * 7) + ( r'\correctchoice{' if c[1] else r'\wrongchoice{' } + c[0] + '}\n'
```

1. Formatos para AMC

```

def CuestionesJuntas(l):
    def CreaLista(t):
        return t if isinstance(t, list) else [t]
    p = []
    for e in l:
        if isinstance(e, str):
            p.append(e)
        elif not isinstance(CreaLista(e)[0], Question):
            p.append(e)
        else:
            p.extend(CreaLista(e))
    return p

```

```

def AMC (nombre, etiqueta, enunciado, cuestiones, opc=["", "Ninguna de las anteriores"]):
    InstruccionesAux = opc[0]
    OpcPorDefecto = opc[1]
    s = '\\element{' + nombre + '}' + InstruccionesAux + '\\n'
    s = s + '\\begin{questionmult}' + nombre + '-' + str(etiqueta) + '\\n'
    s = s + ' ' + enunciado + '\\n'

    s = s + '\\begin{choices}\\n'
    for c in cuestiones:
        if c[2]:
            s = s + ' ' + '\\correctchoice{' if c[1] else '\\wrongchoice {'} + c[0] + '\\n'
        s = s + '\\end{choices}\\n'

    s = s + '\\end{questionmult}'
    s = s + '\\n\\n'
    return s

```

```

def AMC_VF (nombre, etiqueta, enunciado, cuestiones, opc=["", "Ninguna de las anteriores"]):
    InstruccionesAux = opc[0]
    OpcPorDefecto = opc[1]
    s = '\\element{' + nombre + '}' + InstruccionesAux + '\\n'
    s = s + '\\begin{questionmult}' + nombre + '-' + str(etiqueta) + '\\n'
    s = s + ' ' + enunciado + '\\n'

    s = s + '\\begin{choices}\\n'
    for c in cuestiones:
        s = s + (" " * 7) + ( '\\correctchoice{' if c[1] else '\\wrongchoice{' } + c[0] + '\\n'
    s = s + '\\end{choices}\\n'

    s = s + '\\end{questionmult}'
    s = s + '\\n\\n'
    return s

```

```

def AMCProfe (nombre, etiqueta, enunciado, cuestiones, opc=["", "Ninguna de las anteriores"]):
    InstruccionesAux = opc[0]
    OpcPorDefecto = opc[1]
    ex = ''
    s = '\\element{' + nombre + '}' + InstruccionesAux + '\\n'
    s = s + '\\begin{questionmult}' + nombre + '-' + str(etiqueta) + '\\n'
    s = s + ' ' + enunciado + '\\n'

    s = s + '\\begin{choices}\\n'
    for c in cuestiones:
        if c[2]:
            s = s + ' ' + '\\correctchoice{' if c[1] else '\\wrongchoice {'} + c[0] + '\\n'
        else:
            ex = ex + ' cuestion: ' + c[0] + '; ' + c[1] + '\\n'
    s = s + '\\end{choices}\\n'
    if ex:
        s = s + '\\explain{' + ex + '}' + '\\n'
    s = s + '\\end{questionmult}'
    s = s + '\\n\\n'
    return s

```

```

def AMClasCh (nombre, etiqueta, enunciado, cuestiones, opc=["", "Ninguna de las anteriores"]):
    InstruccionesAux = opc[0]
    OpcPorDefecto = opc[1]
    s = '\\element{' + nombre + '}' + InstruccionesAux + '\\n'
    s = s + '\\begin{questionmult}' + nombre + '-' + str(etiqueta) + '\\n'
    s = s + ' ' + enunciado + '\\n'

    s = s + '\\begin{choices}\\n'
    for c in cuestiones:

```

```

        if c[2]:
            s = s + ' ' \
                + ('\\correctchoice{' if c[1] else '\\wrongchoice {'} + c[0] + '}'\n'
s = s + ' ' + '\\lastchoices\n'
s = s + ' ' \
        + ('\\wrongchoice {'if any([c[1] for c in cuestiones])else'\\correctchoice{'}\n'
        + OpcPorDefecto + '}'\n'
s = s + ' ' '\\end{choices}\n'

s = s + ' ' '\\end{questionmult} '
s = s + '}'\n\n'
return s

```

```

def AMCmc (nombre, etiqueta, enunciado, cuestiones, c, opc=["", "Ninguna de las anteriores"]):
    InstruccionesAux = opc[0]
    OpcPorDefecto = opc[1]
    s = '\\element{' + nombre + '}' + InstruccionesAux + '\n'
    s = s + ' ' '\\begin{questionmult}' + nombre + '-' + str(etiqueta) + '}'\n'
    s = s + ' ' + enunciado + '\n'
    s = s + ' ' '\\begin{multicols}' + str(c) + '}'\\AMCBoxedAnswers\n'
    s = s + ' ' '\\begin{choices}\n'
    for c in cuestiones:
        if c[2]:
            s = s + ' ' \
                + ('\\correctchoice{' if c[1] else '\\wrongchoice {'} + c[0] + '}'\n'
s = s + ' ' '\\end{choices}\n'
s = s + ' ' '\\end{multicols}\n'
s = s + ' ' '\\end{questionmult} '
s = s + '}'\n\n'
return s

```

```

def AMCmcProfe (nombre, etiqueta, enunciado, cuestiones, c, opc=["", "Ninguna de las anteriores"]):
    InstruccionesAux = opc[0]
    OpcPorDefecto = opc[1]
    ex = ''
    s = '\\element{' + nombre + '}' + InstruccionesAux + '\n'
    s = s + ' ' '\\begin{questionmult}' + nombre + '-' + str(etiqueta) + '}'\n'
    s = s + ' ' + enunciado + '\n'
    s = s + ' ' '\\begin{multicols}' + str(c) + '}'\\AMCBoxedAnswers\n'
    s = s + ' ' '\\begin{choices}\n'
    for c in cuestiones:
        if c[2]:
            s = s + ' ' \
                + ('\\correctchoice{' if c[1] else '\\wrongchoice {'} + c[0] + '}'\n'
            else:
                ex = ex + ' cuestion: ' + c[0] + '; ' + c[1] + '\n'
s = s + ' ' '\\end{choices}\n'
s = s + ' ' '\\end{multicols}\n'
if ex:
    s = s + ' ' '\\explain{' + ex + ' ' + '}'\n'
s = s + ' ' '\\end{questionmult} '
s = s + '}'\n\n'
return s

```

```

def AMClastChmc (nombre, etiqueta, enunciado, cuestiones, c, opc=["", "Ninguna de las anteriores"]):
    InstruccionesAux = opc[0]
    OpcPorDefecto = opc[1]
    s = '\\element{' + nombre + '}' + InstruccionesAux + '\n'
    s = s + ' ' '\\begin{questionmult}' + nombre + '-' + str(etiqueta) + '}'\n'
    s = s + ' ' + enunciado + '\n'
    s = s + ' ' '\\begin{multicols}' + str(c) + '}'\\AMCBoxedAnswers\n'
    s = s + ' ' '\\begin{choices}\n'
    for c in cuestiones:
        if c[2]:
            s = s + ' ' \
                + ('\\correctchoice{' if c[1] else '\\wrongchoice {'} + c[0] + '}'\n'
s = s + ' ' + '\\lastchoices\n'
s = s + ' ' \
        + ('\\wrongchoice {'if any([c[1] for c in cuestiones])else'\\correctchoice{'}\n'
        + OpcPorDefecto + '}'\n'
s = s + ' ' '\\end{choices}\n'
s = s + ' ' '\\end{multicols}\n'
s = s + ' ' '\\end{questionmult} '
s = s + '}'\n\n'
return s

```

13.2. Ejemplo de uso

Al ejecutar `python3 EjemploVFamc.py` donde el fichero `EjemploVFamc.py` contiene el siguiente código

```
from prop import *
from FormatoPreguntas import *

enunciado = "Indique qué afirmaciones son verdaderas:"
banco = [
    ("Todo alumno que va a clase aprueba", False),
    ("Todo alumno que suspende va a clase", False),
    ("Todo alumno que sabe aprueba", True),
    ("Si aprueban todos eres buen profesor", False),
    ("Si suspenden todos eres mal profesor", False),
    ("Si suspenden todos es frustrante", True),]
GenVar = iter( ProblemaVF (enunciado, banco, 3) ) # tres preguntas por variante

nombre = "EjemploVF"
directorio = "../ejemplos/"
with open(directorio + nombre + ".tex","w") as f:
    for i in range(4): # cuatro variantes
        var = (next(GenVar))
        f.write(AMC_VF(nombre,var[0],var[1],var[2]))
```

obtenemos el siguiente fichero `EjemploVF.tex` de problemas Verdadero/Falso en el directorio `../ejemplos`

```
\element{EjemploVF}{

\begin{questionmult}{EjemploVF-1}
Indique qué afirmaciones son verdaderas:
\begin{choices}
\correctchoice{Todo alumno que sabe aprueba}
\wrongchoice {Si aprueban todos eres buen profesor}
\wrongchoice {Todo alumno que suspende va a clase}
\lastchoices
\wrongchoice {Ninguna de las anteriores}
\end{choices}
\end{questionmult} }

\element{EjemploVF}{

\begin{questionmult}{EjemploVF-2}
Indique qué afirmaciones son verdaderas:
\begin{choices}
\correctchoice{Si suspenden todos es frustrante}
\wrongchoice {Todo alumno que suspende va a clase}
\correctchoice{Todo alumno que sabe aprueba}
\lastchoices
\wrongchoice {Ninguna de las anteriores}
\end{choices}
\end{questionmult} }

\element{EjemploVF}{

\begin{questionmult}{EjemploVF-3}
Indique qué afirmaciones son verdaderas:
\begin{choices}
\correctchoice{Todo alumno que sabe aprueba}
\wrongchoice {Si suspenden todos eres mal profesor}
\wrongchoice {Si aprueban todos eres buen profesor}
\lastchoices
\wrongchoice {Ninguna de las anteriores}
\end{choices}
\end{questionmult} }

\element{EjemploVF}{

\begin{questionmult}{EjemploVF-4}
Indique qué afirmaciones son verdaderas:
\begin{choices}
\wrongchoice {Todo alumno que va a clase aprueba}
\correctchoice{Si suspenden todos es frustrante}
```

```

\wrongchoice {Si suspenden todos eres mal profesor}
\lastchoices
\wrongchoice {Ninguna de las anteriores}
\end{choices}
\end{questionmult} }

\element{EjemploVF}{

\begin{questionmult}{EjemploVF-5}
Indique qué afirmaciones son verdaderas:
\begin{choices}
\wrongchoice {Todo alumno que suspende va a clase}
\wrongchoice {Si aprueban todos eres buen profesor}
\wrongchoice {Todo alumno que va a clase aprueba}
\lastchoices
\correctchoice{Ninguna de las anteriores}
\end{choices}
\end{questionmult} }

```

Auto-Multiple-Choice escogerá para cada examen individual una de las variantes al azar (de esta manera es posible que cada alumno tenga un examen distinto al de los demás si generamos un número suficiente de variantes de cada problema). Un examen individual y un catálogo con las cinco variantes se puede ver en el subdirectorio de ejemplos.

1. Ejemplo de uso de una `PreguntaTipo`.

Al ejecutar con Python el código del siguiente recuadro, se genera el fichero `Ejemploamc.tex` con las distintas variantes del `ProblemaTipo` (para ser usadas con [AMC](#)).

```

from prop import *
from FormatoPreguntas import *

p = [
[r"Se dice que un proceso estocástico  $\boldsymbol{X}$  sin componentes deterministas es: "],
[Supuesto(" ", unoDe(v("I(0)"), v("I(1)"), v("I(2)")))],
[
Supuesto(r"$I(0)$ ", v("I(0)")),
Supuesto(r"$I(1)$ ", v("I(1)")),
Supuesto(r"$I(2)$ ", v("I(2)")),
],
[r"cuando"],
[
Cuestion("tiene representación ARMA estacionaria e invertible", v("I(0)")),
Cuestion(r"${1-\mathsf{B}}^0\boldsymbol{X}$ tiene representación ARMA estacionaria e invertible", v("I(0)")),
Cuestion(r"${1-\mathsf{B}}*\boldsymbol{X}$ es I(0)", v("I(1)"), -v("I(0)")),
],
[
Cuestion(r"${1-\mathsf{B}}*\boldsymbol{X}$ tiene representación ARMA estacionaria e invertible", v("I(1)")),
Cuestion(r"$\text{Var}(X_t) < \infty$, para $t \in \mathbb{Z}$", False),
Cuestion(r"${1-\mathsf{B}}^2\boldsymbol{X}$ tiene representación ARMA estacionaria e invertible", v("I(2)")),
],
[
Cuestion(r"${1-\mathsf{B}}^2\boldsymbol{X}$ es I(1)", False),
Cuestion(r"${1-\mathsf{B}}*\boldsymbol{X}$ es I(1)", v("I(2)")),
Cuestion(r"${1-\mathsf{B}}^2\boldsymbol{X}$ es I(0)", v("I(2)"), -v("I(0)")),
],
],
preguntas = {}; mc=1;
preguntas[nombre] = ProblemaTipoProfe( p );
nombre = "Ejemploamc"
directorio = "../ejemplos/"

mc = 1 # preguntas en una columna
with open(directorio + nombre + ".tex", "w") as f:
for var in preguntas[nombre]:
f.write( AMCmcProfe(nombre, var[0], var[1], var[2], mc) )

```

```

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-1}
Considere A y que A implica C. Indique qué opción es correcta:
\begin{multicols}{2}\AMCBoxedAnswers

```

```

\begin{choices}
\correctchoice{Entonces C es verdadero}
\end{choices}
\end{multicols}
\end{questionmult} }

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-2}
Considere A y que A implica C. Indique qué opción es correcta:
\begin{multicols}{2}\AMCBoxedAnswers
\begin{choices}
\wrongchoice {Entonces D es verdadero}
\end{choices}
\end{multicols}
\end{questionmult} }

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-3}
Considere A y que B equivale a D. Indique qué opción es correcta:
\begin{multicols}{2}\AMCBoxedAnswers
\begin{choices}
\wrongchoice {Entonces C es verdadero}
\end{choices}
\end{multicols}
\end{questionmult} }

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-4}
Considere A y que B equivale a D. Indique qué opción es correcta:
\begin{multicols}{2}\AMCBoxedAnswers
\begin{choices}
\wrongchoice {Entonces D es verdadero}
\end{choices}
\end{multicols}
\end{questionmult} }

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-5}
Considere B y que A implica C. Indique qué opción es correcta:
\begin{multicols}{2}\AMCBoxedAnswers
\begin{choices}
\wrongchoice {Entonces C es verdadero}
\end{choices}
\end{multicols}
\end{questionmult} }

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-6}
Considere B y que A implica C. Indique qué opción es correcta:
\begin{multicols}{2}\AMCBoxedAnswers
\begin{choices}
\wrongchoice {Entonces D es verdadero}
\end{choices}
\end{multicols}
\end{questionmult} }

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-7}
Considere B y que B equivale a D. Indique qué opción es correcta:
\begin{multicols}{2}\AMCBoxedAnswers
\begin{choices}
\wrongchoice {Entonces C es verdadero}
\end{choices}
\end{multicols}

```

```

\end{questionmult} }

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-8}
  Considere B y que B equivale a D. Indique qué opción es correcta:
  \begin{multicols}{2}\AMCBoxedAnswers
    \begin{choices}
      \correctchoice{Entonces D es verdadero}
    \end{choices}
  \end{multicols}
\end{questionmult} }

\element{Ejemploamc}{

\begin{questionmult}{Ejemploamc-9}
  Considere B y que B equivale a D. Indique qué opción es correcta:
  \begin{multicols}{2}\AMCBoxedAnswers
    \begin{choices}
      \wrongchoice {Entonces E es verdadero}
    \end{choices}
  \end{multicols}
\end{questionmult} }

```

Capítulo 14

Preguntas en xml para usar con Moodle

Para generar los ficheros xml vamos a seguir un método indirecto. El motivo es que escribir en xml puede ser tedioso debido a los caracteres no ASCII y las expresiones matemáticas de L^AT_EX. La solución más práctica que he encontrado es generar desde Python un fichero L^AT_EX que a su vez emplea el paquete [moodle](#). Así, al compilar el fichero de L^AT_EX con `pdflatex` obtendremos, tanto una versión en pdf de las preguntas (¡Algo mucho más cómodo a la hora de trabajar), como un fichero xml para ser importado en Moodle. Además, este método indirecto nos evita generar la estructura de los ficheros de preguntas en xml al estilo Moodle... ¡la estructura en L^AT_EX es mucho más sencilla! (o al menos, más familiar).

```
s = "\\documentclass[11pt]{article}\\n\\n"
s = s + "\\usepackage[cm,headings]{fullpage}\\n\\n"
s = s + "\\usepackage{moodle}\\n\\n"
s = s + "\\usepackage{graphicx}\\n\\n"
s = s + "\\usepackage{fancyvrb}\\n\\n"
s = s + "\\ifPDFTeX                % FOR LATEX and PDFLATEX\\n"
s = s + "    \\usepackage[utf8]{inputenc}    % necessary\\n"
s = s + "    \\usepackage[OT1]{fontenc}      % necessary\\n"
s = s + "\\else                % assuming XELATEX or LUALATEX\\n"
s = s + "    \\usepackage{fontspec}\\n"
s = s + "\\fi\\n\\n"
s = s + "\\auxLaTeX + \"\\n\" + \"\\newcommand\\peque{}\" + \"\\n\\n\"
s = s + "\\begin{document}\\n\\n"
```

Con las cuestiones de opción múltiple, el 100\\pregunta se obtiene si se marcan todas las correctas (indicadas con `\\item*` y el 100\\incorrectas (las que aparecen con `=`)).

Advertencia: Lamentablemente Moodle no permite que la puntuación global de ninguna pregunta de opción múltiple sea negativa (es imposible que las preguntas de opción múltiple tengan esperanza nula; en contraste con [AMC](#), donde se puede programar cualquier regla de puntuación).

```
for c in cuestiones:
    #s = s + "                " + ("\\item* " if c[1] else "\\item ") + codchar(c[0]) + "\\n"
    s = s + "                " + ( itemBuena(c[3]) if c[1] else itemMala(c[3]) ) + codchar(c[0]) + "\\n"

for c in cuestiones:
    if c[2]:
        s = s + '                ' + (itemBuena(b,c[3]) if c[1] else itemMala(m,c[3])) + codchar(c[0]) + "\\n"
        #s = s + '                ' + (buena if c[1] else mala) + codchar(c[0]) + "\\n"
    else:
        ex = ex + ' cuestion: ' + c[0] + ' ; ' + c[1] + '\\n'
```

Si ninguna respuesta es correcta, al compilar el fichero de L^AT_EX obtendremos un error (Moodle exige que al menos haya una respuesta correcta). Por ello, podemos incluir la opción `{“Las demás respuestas son incorrectas”}`. Lamentablemente, Moodle tampoco tiene un comando del tipo `\\lastchoice`, que

fuerza a que esta opción aparezca en último lugar aunque se barajen las opciones; por eso pongo *las demás* en lugar de *las anteriores* como en los exámenes con [AMC](#). Esta opción será falsa si alguna de las demás es True.

```
v = [c[1] for c in cuestiones].count(True)
cuestiones = cuestiones + [(codchar(lastchoice), (False if v else True), 1, '')]
```

14.1. QuizMoodle, QuizMoodleLastCh, QuizVFMoodle y QuizVFMoodleLastCh

QuizMoodle genera un archivo $\text{\LaTeX}$ con las variantes de un ProblemaTipo. Sus argumentos son el nombre del ProblemaTipo, el directorio donde queremos crear el fichero de $\text{\LaTeX}$, y la lista problema que contiene las variantes. La cadena `s` contiene el preámbulo del archivo. A la cadena `s` se le concatena el inicio del entorno `quiz`, cuyo primer argumento es el nombre del problema (del que escribiremos las distintas versiones).

- Con `with open()` creamos el archivo `nombre.tex` en el directorio que hayamos indicado; y entonces
 - Escribimos la cadena `s` en dicho archivo.
 - Luego recorreremos todas las variantes de la lista `=problema` con un `for`. Por cada variante llamamos a `MoodleMulti` para escribir una pregunta de opción múltiple con los argumentos `nombre`, número de variante (`var[0]`), enunciado de la variante (`var[1]`), y las cuestiones de la variante (`var[2]`).
 - Una vez terminado el bucle `for`, escribimos el cierre del entorno `quiz` y el final del documento de $\text{\LaTeX}$.

```
def creaDiccionario(x, key='key'):
    return x if isinstance(x, dict) else {key: x}
```

```
def QuizMoodle (nombre, directorio, problema, opc=["", ""]):
    <<Método auxiliar que transforma un objeto en diccionario si no era ya un diccionario>>
    problema = creaDiccionario(problema, nombre)
    auxLaTeX = opc[0]
    <<Preámbulo fichero \LaTeX{} con paquete moodle>>
    s = s + "\begin{quiz}{" + nombre + "}\n\n"
    with open(directorio + nombre + ".tex", "w") as f:
        f.write(s)
        for i,nom in enumerate(problema):
            for var in problema[nom]:
                f.write( MoodleMulti(codchar(nom), var[0], codchar(var[1]), var[2]) )
        f.write("\end{quiz}\n\n\end{document}\n")
```

```
def QuizMoodleConVariables (nombre, directorio, problema, environment, Valores, opc=["", ""]):
    <<Método auxiliar que transforma un objeto en diccionario si no era ya un diccionario>>
    problema = creaDiccionario(problema, nombre)
    auxLaTeX = opc[0]
    <<Preámbulo fichero \LaTeX{} con paquete moodle>>
    s = s + "\begin{quiz}{" + nombre + "}\n\n"
    with open(directorio + nombre + ".tex", "w") as f:
        f.write(s)
        for i,nom in enumerate(problema):
            for var in problema[nom]:
                template = environment.from_string(MoodleMulti(codchar(nom), var[0], codchar(var[1]), var[2]))
                content = template.render(Valores())
                f.write(content)
                #f.write( MoodleMulti(codchar(nom), var[0], codchar(var[1]), var[2]) )
        f.write("\end{quiz}\n\n\end{document}\n")
```

```
def QuizMoodleProfe (nombre, directorio, problema, opc=["", ""]):
    <<Método auxiliar que transforma un objeto en diccionario si no era ya un diccionario>>
    problema = creaDiccionario(problema, nombre)
    auxLaTeX = opc[0]
    <<Preámbulo fichero \LaTeX{} con paquete moodle>>
    s = s + "\begin{quiz}{" + nombre + "}\n\n"
    with open(directorio + nombre + ".tex", "w") as f:
```

```

f.write(s)
for i,nom in enumerate(problema):
    for var in problema[nom]:
        f.write( MoodleMultiProfe(codchar(nom), var[0], codchar(var[1]), var[2]) )
f.write("\end{quiz}\n\n\end{document}\n")

```

```

def QuizMoodleProfeConVariables (nombre, directorio, problema, environment, Valores, opc=["", ""]):
    <<Método auxiliar que transforma un objeto en diccionario si no era ya un diccionario>>
    problema = creaDiccionario(problema, nombre)
    auxLaTeX = opc[0]
    <<Preámbulo fichero \LaTeX{} con paquete moodle>>
    s = s + "\\begin{quiz}{" + nombre + "}" + "\n\n"
    with open(directorio + nombre + ".tex", "w") as f:
        f.write(s)
        for i,nom in enumerate(problema):
            for var in problema[nom]:
                template = environment.from_string(MoodleMultiProfe(codchar(nom), var[0], codchar(var[1]), var[2]))
                content = template.render(Valores())
                f.write(content)
                #f.write( MoodleMultiProfe(codchar(nom), var[0], codchar(var[1]), var[2]) )
        f.write("\end{quiz}\n\n\end{document}\n")

```

Si necesitamos que las variantes añadan la [cuestion alternativa por defecto para Moodle](#) para así asegurarnos de que siempre hay una respuesta correcta, entonces empleamos el método `QuizMoodleLastCh`, que hace lo mismo que el anterior pero llamando a `MoodleMultiLastCh`.

```

def QuizMoodleLastCh (nombre, directorio, problema, opc=["", "Las demás opciones son falsas"]):
    <<Método auxiliar que transforma un objeto en diccionario si no era ya un diccionario>>
    auxLaTeX = opc[0]
    OpcPorDefecto = opc[1]
    <<Preámbulo fichero \LaTeX{} con paquete moodle>>
    s = s + "\\begin{quiz}{" + nombre + "}" + "\n\n"
    with open(directorio + nombre + ".tex", "w") as f:
        f.write(s)
        for i,nom in enumerate(creaDiccionario(problema, nombre)):
            for var in problema[nom]:
                f.write(MoodleMultiLastCh(codchar(nom), var[0], codchar(var[1]), var[2], opc[1]))
        f.write("\end{quiz}\n\n\end{document}\n")

```

```

def QuizMoodleLastChConVariables (nombre, directorio, problema, environment, Valores, opc=["", "Las demás opciones son falsas"]):
    <<Método auxiliar que transforma un objeto en diccionario si no era ya un diccionario>>
    auxLaTeX = opc[0]
    OpcPorDefecto = opc[1]
    <<Preámbulo fichero \LaTeX{} con paquete moodle>>
    s = s + "\\begin{quiz}{" + nombre + "}" + "\n\n"
    with open(directorio + nombre + ".tex", "w") as f:
        f.write(s)
        for i,nom in enumerate(creaDiccionario(problema, nombre)):
            for var in problema[nom]:
                template = environment.from_string(MoodleMultiLastCh(codchar(nom), var[0], codchar(var[1]), var[2]))
                content = template.render(Valores())
                f.write(content)
                #f.write(MoodleMultiLastCh(codchar(nom), var[0], codchar(var[1]), var[2], opc[1]))
        f.write("\end{quiz}\n\n\end{document}\n")

```

`QuizVFMoodle` genera un archivo $\text{\LaTeX}$ con las variantes de un `ProblemaVF`. Los argumentos son el nombre del `ProblemaVF`, el directorio donde crear el archivo, el iterador `GenVar` que genera las variantes, y el número de variantes que se desea. Como antes, la cadena `s` contiene el preámbulo del archivo y el inicio del entorno `quiz`, cuyo primer argumento es el nombre del problema.

- Con `with open()` creamos el archivo `nombre.tex` en el directorio que hayamos indicado; y entonces
 - escribimos la cadena `s` en dicho archivo.
 - luego iteramos el bucle `for` tanta veces como hayamos indicado en `num`. En cada iteración obtenemos una nueva variante `var`, llamamos a `MoodleMulti` para escribir una pregunta de opción múltiple con los argumentos `nombre`, número de variante (`var[0]`), enunciado de la variante (`var[1]`), y las cuestiones de la variante (`var[2]`).

- una vez terminado el bucle `for`, escribimos el cierre del entorno `quiz` y el final del documento de $\text{\LaTeX}$.

```
def QuizVFMoodle (nombre, directorio, GenVar, num, opc=["", ""]):
    auxLaTeX = opc[0]
    <<Preámbulo fichero \LaTeX{} con paquete moodle>>
    s = s + "\\begin{quiz}{ " + nombre + " }\n\n"
    with open(directorio + nombre + ".tex", "w") as f:
        f.write(s)
        for i in range(num):
            var = next(GenVar)
            f.write( MoodleMulti (codchar(nombre), var[0], codchar(var[1]), var[2]) )
        f.write("\\end{quiz}\n\n\\end{document}\n")
```

Cuando necesitamos que las variantes añadan la [cuestion alternativa por defecto para Moodle](#), empleamos el método `QuizVFMoodleLastCh`, que hace lo mismo que el anterior pero llamando a `MoodleMultiLastCh`.

```
def QuizVFMoodleLastCh (nombre, directorio, GenVar, num, opc=["", "Las demás opciones son falsas"]):
    auxLaTeX = opc[0]
    <<Preámbulo fichero \LaTeX{} con paquete moodle>>
    s = s + "\\begin{quiz}{ " + nombre + " }\n\n"
    with open(directorio + nombre + ".tex", "w") as f:
        f.write(s)
        for i in range(num):
            var = next(GenVar)
            f.write(MoodleMultiLastCh(codchar(nombre), var[0], codchar(var[1]), var[2], opc[1]))
        f.write("\\end{quiz}\n\n\\end{document}\n")
```

14.2. MoodleMulti y MoodleMultiLastCh

`MoodleMulti` genera una cadena de caracteres con los comandos $\text{\LaTeX}$ correspondientes a una pregunta de opción múltiple. Los parámetros del método `MoodleMulti` son: `nombre` (el nombre del problema) y `variante` (el número de la variante)¹, el enunciado de la variante y la lista de `cuestiones` de la variante.

```
def MoodleMulti (nombre, variante, enunciado, cuestiones):
    <<Escritura del entorno multi>>
```

El método `MoodleMulti` escribe el código $\text{\LaTeX}$ de un entorno `multi` del paquete `moodle` con los siguientes parámetros opcionales para dicho entorno: `multiple` para que sea una pregunta de opción múltiple y `points` el número de opciones de respuesta de la pregunta (la longitud de `cuestiones`).

```
<<Escritura de cada item de cuestión en Moodle>>

ex = ''
s = " \\begin{multi}[multiple, points=" + str(len(cuestiones)-1) + "]"
s = s + "{ " + codchar(nombre) + "-" + str(variante) + " }\n\n"
s = s + " " + enunciado + "\n\n"

<<Escritura de las cuestiones para Moodle>>

s = s + " \\end{multi}\n\n"
return s
```

```
def MoodleMultiProfe (nombre, variante, enunciado, cuestiones):
    <<Escritura aclaración en Moodle>>
    <<Escritura del entorno multi para el Profe>>
```

```
def feedback(texto):
    return r",feedback={" + codchar(texto) + "}"
```

```
<<Conteo de respuestas correctas e incorrectas>>
<<Escritura de cada item de cuestión en Moodle para Profe>>
```

```
ex = ''
```

¹Que separados por un guión son empleados como nombre de cada variante del problema de opción múltiple dentro del código $\text{\LaTeX}$.

```
s = " \\begin{multi}[multiple, fractiontol=5.1" + "]" # , points=" + str(len(cuestiones)-1)
s = s + "{ " + codchar(nombre) + "-" + str(variante) + "}"\n"
s = s + " " + enunciado + "\n"
```

```
<<Escritura de las cuestiones para Moodle para el Profe>>
```

```
s = s + " \\end{multi}\n\n"
return s
```

```
b = 0; m = 0;
for c in cuestiones:
    print(c)
    if c[2]:
        if c[1]:
            b+=1
        else:
            m+=1
```

```
def itemBuena(aclaracion):
    return ("\\item[feedback={" + codchar(aclaracion) + "}]* ") if aclaracion else r"\\item* "
```

```
def itemMala(aclaracion):
    return ("\\item[feedback={" + codchar(aclaracion) + "}] ") if aclaracion else r"\\item "
```

```
def itemBuena(numBuenas,aclaracion):
    return ("\\item[fraction=" + str( round(100/numBuenas) ) + feedback(aclaracion) + "]" if numBuenas else "\\item*"
```

```
def itemMala(numMalas,aclaracion):
    return ("\\item[fraction=" + str(-round(100/numMalas) ) + feedback(aclaracion) + "]" if numMalas else "\\item"
```

Cuando requerimos añadir la [cuestion alternativa por defecto para Moodle](#) llamamos a MoodleMultiLastCh que añade la opción “*Las demás opciones son falsas*” a la lista cuestiones.

```
def MoodleMultiLastCh (nombre, variante, enunciado, cuestiones, \
                        lastchoice="Las demás opciones son falsas"):
    <<cuestion alternativa por defecto para Moodle>>
    <<Escritura del entorno multi>>
```

14.2.1. Codificación de caracteres no ASCII

```
def codchar(s):
    s = s.replace("á", "\\{'a}")
    s = s.replace("é", "\\{'e}")
    s = s.replace("í", "\\{'i}")
    s = s.replace("ó", "\\{'o}")
    s = s.replace("ú", "\\{'u}")
    s = s.replace("ñ", "\\~{n}")
    return s
```

14.3. Ejemplo de uso de QuizMoodleLastCh y QuizVFMoodleLastCh

```
from prop import *
from FormatoPreguntas import *
p = ProblemaTipo( \
    [ "Considere ",
      [
        Supuesto("$\\mathcal{A}$ ", v("A")),
        Supuesto("$\\mathcal{B}$ ", v("B")),
      ],
      [
        Supuesto("y que $\\mathcal{A}\\rightarrow\\mathcal{C}$ ", \
          v("A") >> v("C")),
        Supuesto("y que $\\mathcal{B}\\rightarrow\\mathcal{D}$ ", \
          v("B") ** v("D")),
      ],
      "Indique qué opción es correcta: ",
      [
        Question("Entonces $\\mathcal{C}$ es verdadero", v("C")),
        Question("Entonces $\\mathcal{D}$ es verdadero", v("D")),
        Question("Entonces $\\mathcal{E}$ es verdadero", v("E"), v("D")),
      ],
    ],
```

```

    ])
nombre      = "EjemploMoodle"
directorio  = "../ejemplos/"
QuizMoodleLastCh (nombre, directorio, p) # con alternativa por defecto

```

```

from prop import *
from FormatoPreguntas import *

enunciado = "Indique qué afirmaciones son verdaderas:"
banco = [
    ("Todo alumno que va a clase aprueba", False),
    ("Todo alumno que suspende va a clase", False),
    ("Todo alumno que sabe aprueba", True),
    ("Si aprueban todos eres buen profesor", False),
    ("Si suspenden todos eres mal profesor", False),
    ("Si suspenden todos es frustrante", True),]

b = [(codchar(p[0]),p[1]) for p in banco]

GenVar = iter( ProblemaVF (codchar(enunciado), b, 3) ) # tres respuestas opcionales
# (sin contar con la alternativa por defecto)

nombre      = "EjemploVFMoodle"
directorio  = "../ejemplos/"

QuizVFMoodleLastCh (nombre, directorio, GenVar, 4) # con alternativa por defecto

```

Parte IV

Ejemplo de pregunta de opción múltiple para Econometría

1. Para Moodle.

Los procedimientos QuizVFMoodleLastCh, QuizMoodleLastCh, QuizVFMoodle y QuizMoodle crean (en el directorio indicado, y con el nombre indicado) el fichero de $\text{\LaTeX}$ directorio/nombre-Moodle.tex. Dicho fichero emplea el paquete `moodle` que he sacado de <https://gitlab.mattgk.myds.me/mattguer/moodle>. Al compilarlo con `pdflatex` obtenemos un fichero pdf para revisar las preguntas (algo muy cómodo a la hora de desarrollar un banco de preguntas). Pero si compilamos el fichero con `lualatex` (o con `xelatex`) obtendremos tanto el fichero pdf como otro fichero xml para ser importado en Moodle. Puestos a facilitar las cosas, el siguiente código de Python crea tanto el fichero xml como el pdf. Para ello importamos el método `call` de la librería `subprocess`, y decimos que compile el fichero .tex con `lualatex` en el directorio indicado.

2. Para Auto-Multiple-Choice.

Los procedimientos AMC, AMClastCh, AMCMc y AMClastChmc generan un string con el código $\text{\LaTeX}$ de las preguntas en el formato `AMC`. Dicho código se escribe en el fichero directorio/nombre.tex, que nos servirá como parte del banco de preguntas a usar con `AMC` para generar exámenes aleatorios.

```

from prop import *
from FormatoPreguntas import *
from subprocess import call
p = ProblemaTipo( \
[
[
"En el ajuste MCO del modelo lineal " \
+ "$\mathbf{y}=\mathbf{X}\mathbf{\beta}+\mathbf{otros}$, donde " \
+ "$\mathop{\mathbf{X}}\limits_{\scriptscriptstyle N\times K}$, "
],
[
Supuesto("si $\mathbf{X} < k$, donde $k \leq N$, entonces: ",
v("Mcoli")),

Supuesto("si $\mathbf{X} = k$, donde $k \leq N$, entonces: ",
-v("Mcoli")),
],
[
Cuestion("Ningún regresor es combinación lineal de los demás",
-v("Mcoli")),

Cuestion("Las $k$ columnas de $\mathbf{X}$ son linealmente indep.",
-v("Mcoli")),

Cuestion("Las $N$ filas de $\mathbf{X}$ son linealmente indep.",
False),

Cuestion("Algún regresor es combinación lineal del resto",
v("Mcoli")),
],
[
Cuestion("El vector $\mathbf{y}$ es combinación lineal de los regresores",
False, -v("Mcoli")),

Cuestion("Las ecuaciones normales tienen una \emph{única} solución",
-v("Mcoli")),

Cuestion("Es sistema de ecuaciones normales es compatible (tiene solución)",
True),
],
[
Cuestion("El estimador MCO $\widehat{\mathbf{\beta}}$ está definido",
-v("Mcoli")),

Cuestion("La correlación entre regresores es 1 en valor absoluto",
False),

Cuestion("La correlación entre regresores es menor que 1 en valor absoluto",
-v("Mcoli")),
],
])

nombre = "EcNormales-RangoX"
directorio = "../ejemplos/"

QuizMoodleLastCh (nombre + "-Moodle", directorio, p)
cmd = "cd " + directorio + "&& lualatex " + nombre + "-Moodle" + ".tex"
status = call(cmd, cwd=directorio, shell=True)

```

```
with open(directorio + nombre + ".tex","w") as f:
    for var in p:
        f.write( AMclastCh(nombre, var[0], var[1], var[2]) )
```

La <<Pregunta EcNormales-RangoX>> contempla dos Supuesto=s excluyentes: que haya multicolinealidad (=v("Mcoli")), o que no (-v("Mcoli")); y cuenta con tres bloques de Cuestion=es excluyentes (más la opción /`ninguna de las anteriores'/' para que siempre haya una opción correcta). Una =Cuestion siempre es True; tres son False y el resto depende del Supuesto en cuestión. Una de ellas se formula solo si\; -v("Mcoli")\; es decir, tiene precondition.

Resultado: 60 variantes de esta pregunta.

```
p = ProblemaTipo( \
[
[ "En el ajuste MCO del modelo lineal " \
+ "$\\mathbf{y}=\\mathbf{X}\\mathbf{\\beta}+\\mathbf{otros}$, donde " \
+ "$\\mathop{\\mathbf{X}}\\limits_{\\scriptscriptstyle N\\times K}$", "
],
[
Supuesto("si $\\mathbf{X}$ < k$, donde $k \\leq N$, entonces: ",
-v("Mcoli")),

Supuesto("si $\\mathbf{X}$ = k$, donde $k \\leq N$, entonces: ",
-v("Mcoli")),
],
[
Cuestion("Ningún regresor es combinación lineal de los demás",
-v("Mcoli")),

Cuestion("Las $k$ columnas de $\\mathbf{X}$ son linealmente indep.",
-v("Mcoli")),

Cuestion("Las $N$ filas de $\\mathbf{X}$ son linealmente indep.",
False),

Cuestion("Algún regresor es combinación lineal del resto",
v("Mcoli")),
],
[
Cuestion("El vector $\\mathbf{y}$ es combinación lineal de los regresores",
False, -v("Mcoli")),

Cuestion("Las ecuaciones normales tienen una \\emph{única} solución",
-v("Mcoli")),

Cuestion("Es sistema de ecuaciones normales es compatible (tiene solución)",
True),
],
[
Cuestion("El estimador MCO $\\widehat{\\mathbf{\\beta}}$ está definido",
-v("Mcoli")),

Cuestion("La correlación entre regresores es 1 en valor absoluto",
False),

Cuestion("La correlación entre regresores es menor que 1 en valor absoluto",
-v("Mcoli")),
],
])
```

```
% #Cuestion("Los regresores son linealmente indep.", -v("Mcoli")),
```

3. Advertencia sobre L^AT_EX dentro de Moodle.

Desgraciadamente Moodle no permite modificar el preámbulo de L^AT_EX salvo si se es el administrador de sistema (en nuestro caso, los servicios informáticos de la UCM). Así que si se quiere usar algún paquete específico de L^AT_EX (por ejemplo `amsmath` para representar un vector en negrita cursiva) no es posible. Y si definimos algún comando de L^AT_EX, no será interpretado por Moodle (es decir, las preguntas tipo test mostrarán el código de dichos los comandos de L^AT_EX pero sin interpretarlo).